



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

00/00/2026

Análisis y Diseño de Sistemas 1

Escuela de Ingeniería de Ciencias Y Sistemas

Tutor: Nombre del tutor

UNIDAD 3:

Requerimientos

Anuncios Importantes



Anuncio 1



Anuncio 2



Anuncio 3



Anuncio 4



Anuncio 5

Agenda



TEMA 1



TEMA 2



TEMA 3



TEMA 4



TEMA 5

COMPETENCIA(S) QUE DESARROLLAREMOS



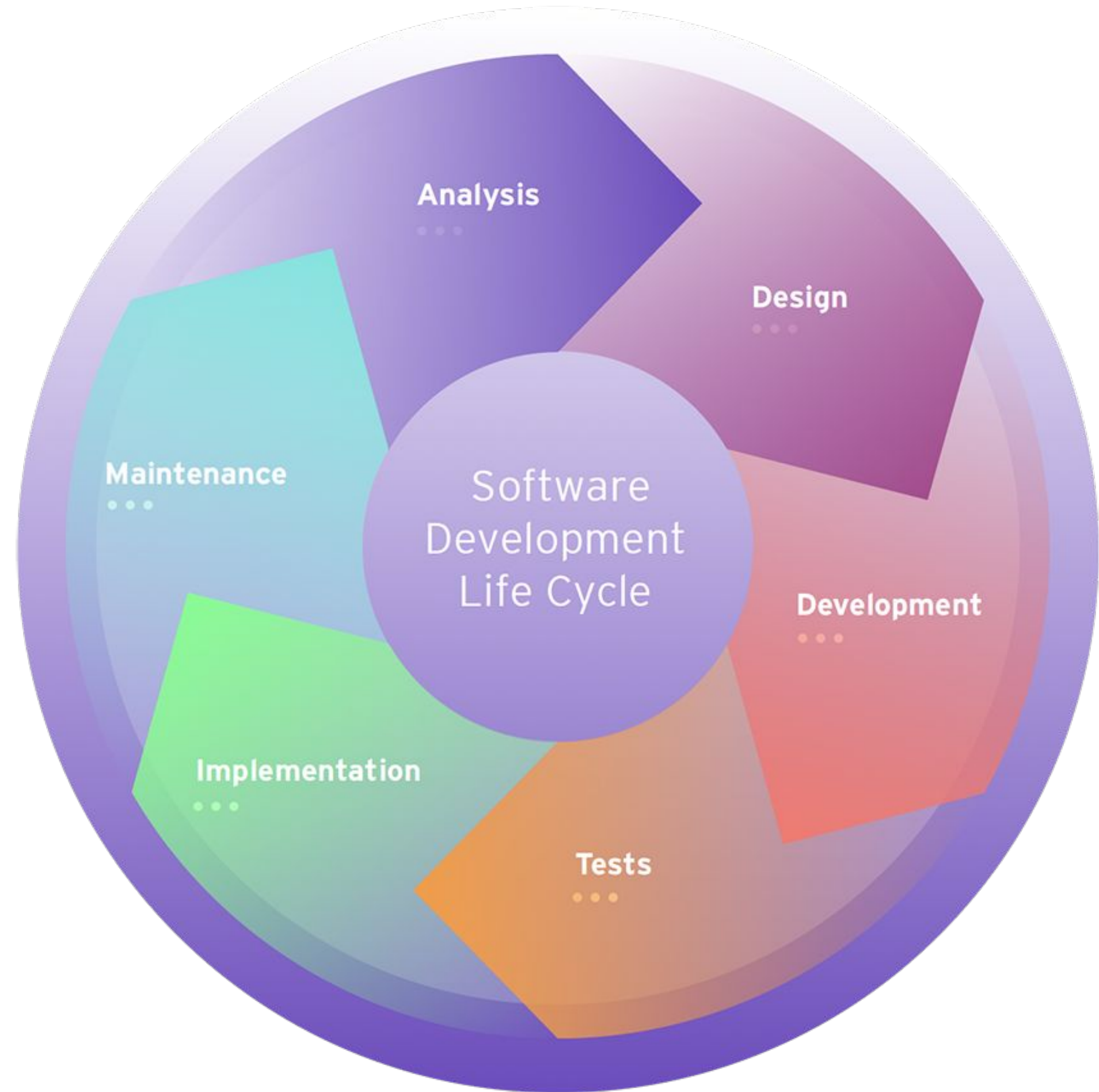
1. El estudiante Analiza proceso de ingeniería de requerimientos
2. El estudiante Identifica requerimientos funcionales de un sistema
3. El estudiante Define requerimientos no funcionales para garantizar calidad
4. El estudiante Crea historias de usuario User Stories con formato estándar
5. El estudiante Establece criterios de aceptación para validar funcionalidades



VALOR DE LA SEMANA: Tolerancia

En la ingeniería de requerimientos, el analista trabaja con múltiples stakeholders que tienen visiones, prioridades y lenguajes distintos. La tolerancia es clave para entender que no hay una sola "versión correcta" del sistema: cada actor aporta una perspectiva válida. Saber tolerar y gestionar esas diferencias es lo que permite construir requerimientos completos, coherentes y aceptados por todos los involucrados.

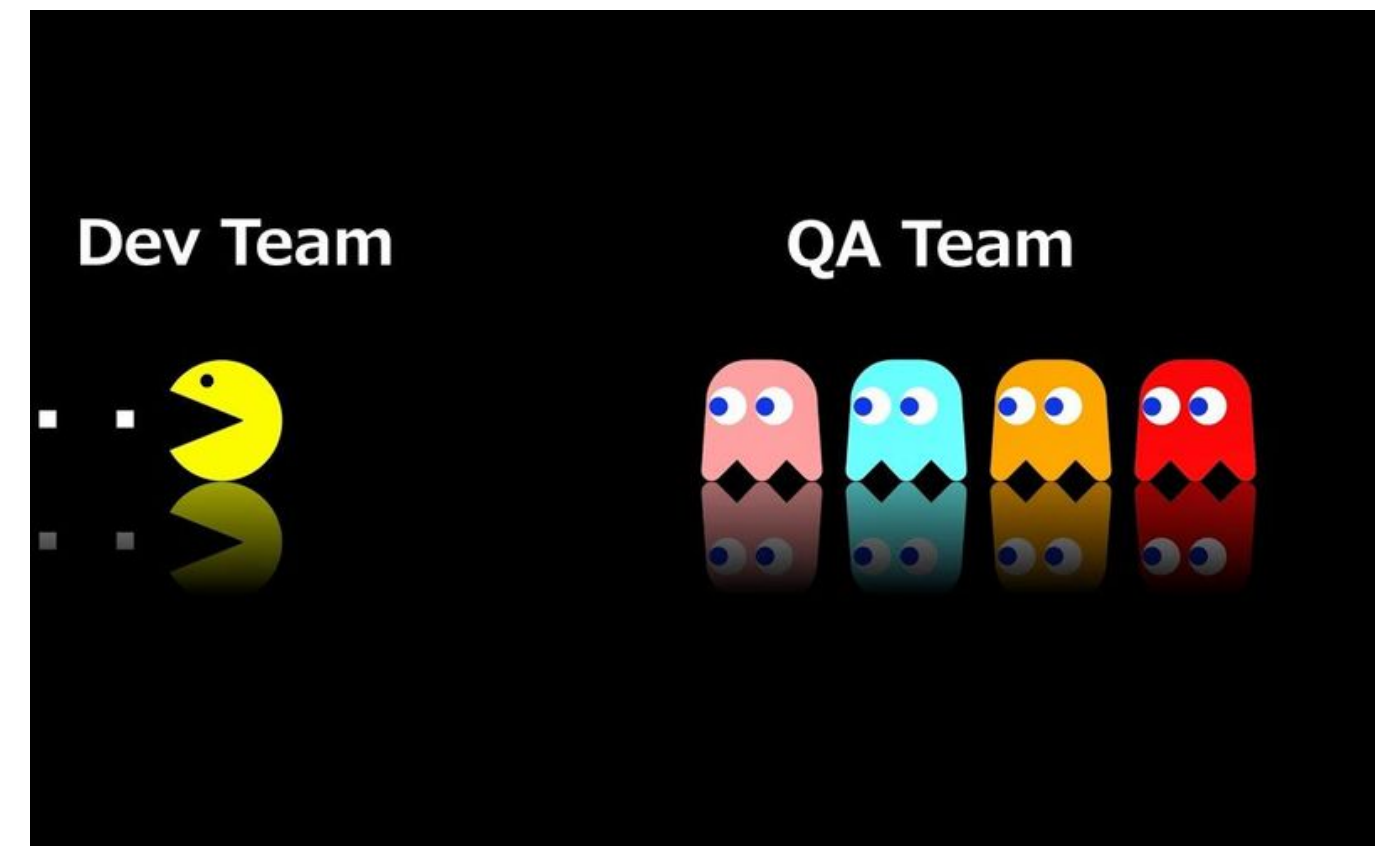
El ciclo de vida del software define las fases por las que pasa un proyecto de software, desde su concepción hasta su retiro, y en cada una de estas fases se aplican actividades de aseguramiento de la calidad para garantizar que el producto final cumpla con los estándares y requisitos establecidos.



EL ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE (SQA)

Es un conjunto de actividades y procesos sistemáticos diseñados para garantizar que el software cumpla con los estándares de calidad establecidos y satisfaga las expectativas del cliente.

El SQA no se limita a la fase de pruebas, sino que abarca todo el ciclo de vida del software, con el objetivo de prevenir defectos en lugar de simplemente detectarlos.



FASE: ANÁLISIS

Se identifican y documentan las necesidades del cliente y los objetivos del software.

El SQA asegura que los requisitos del software estén bien definidos, sean completos, consistentes y medibles.

Requisitos Funcionales
Requisitos No Funcionales

FASE: DISEÑO

Se crea un plan técnico para implementar los requisitos, incluyendo arquitectura y diseño del sistema.

El SQA verifica que el diseño del software sea coherente con los requisitos y que siga buenas prácticas de arquitectura y diseño.

Diseño de la Base de Datos
Diagramas de componentes
Diseño de la Interfaz de Usuario (Figma)

FASE: DESARROLLO

Se escribe el código según el diseño establecido.

El SQA supervisa la escritura del código, asegurando que se sigan estándares de codificación y buenas prácticas.

Principios SOLID
Patrones de diseño

FASE: PRUEBA

El equipo de desarrollo hace pruebas técnicas porque están directamente relacionadas con el código y su implementación.

El equipo de QA hace pruebas funcionales y no funcionales porque se enfocan en validar el software desde la perspectiva del usuario y los requisitos del negocio



El software se implementa en el entorno de producción para su uso.

El equipo de desarrollo se enfoca en tareas técnicas como la configuración del entorno, la implementación del código y la resolución de problemas, mientras que el equipo de QA se encarga de validar que el software funcione correctamente en producción y cumpla con los requisitos del cliente

EQUIPO DEV

- Preparación del entorno de producción
- Implementación del código
- Migración de datos
- Configuración de variables de entorno
- Rollback (en caso de fallos)

EQUIPO QA

- Pruebas de humo (Smoke Testing)
- Pruebas de sanidad (Sanity Testing)
- Validación del entorno de producción
- Pruebas de aceptación en producción
- Reporte de problemas nuevos
- Verificación de seguridad

FASE: MANTENIMIENTO

La fase de mantenimiento es la etapa en la que el software ya está en producción y se realizan actividades para asegurar que continúe funcionando correctamente, se adapte a nuevos requisitos y se corrijan problemas que surjan con el tiempo. Esta fase es crítica porque el software no es estático sino que evoluciona con el tiempo

EQUIPO DEV

- Mantenimiento Correctivo
- Mantenimiento Adaptativo
- Mantenimiento Preventivo
- Actualización de Dependencias

EQUIPO QA

- Pruebas de Regresión
- Pruebas No Funcionales
- Validación de Correcciones
- Pruebas de Nuevas Funcionalidades

METODOLOGÍAS ÁGILES EN EL CICLO DE VIDA DEL SOFTWARE

FASE: ANÁLISIS

Enfoque tradicional: Los requisitos se definen completamente al inicio del proyecto

Enfoque ágil: Los requisitos se capturan en forma de historias de usuario y se priorizan en un backlog. Los requisitos pueden evolucionar con el tiempo, y se ajustan en cada iteración según el feedback del cliente.

FASE: DISEÑO

Enfoque tradicional: El diseño se realiza completamente antes de comenzar la implementación.

Enfoque ágil: El diseño se hace de manera incremental. En cada iteración, se diseña solo lo necesario para las funcionalidades que se van a desarrollar en ese sprint.

FASE: DESARROLLO

Enfoque tradicional: La implementación se realiza en una sola fase, después del diseño.

Enfoque ágil: La implementación se realiza en iteraciones cortas (sprints), donde se desarrollan pequeñas funcionalidades completas. El equipo de desarrollo trabaja en colaboración con el equipo de QA para garantizar la calidad del código.

FASE: PRUEBAS

Enfoque tradicional: Las pruebas se realizan después de completar la implementación.

Enfoque ágil: Las pruebas se integran en cada iteración. El equipo de QA trabaja en paralelo con los desarrolladores, realizando pruebas continuas (unitarias, de integración, funcionales, etc.). Además, se fomenta la automatización de pruebas para agilizar el proceso.

FASE: DESPLIEGUE

Enfoque tradicional: El despliegue se realiza al final del proyecto, después de completar todas las fases.

Enfoque ágil: El despliegue puede realizarse de manera continua o al final de cada iteración, dependiendo del proyecto. Esto permite entregar valor al cliente de manera frecuente y obtener feedback temprano.

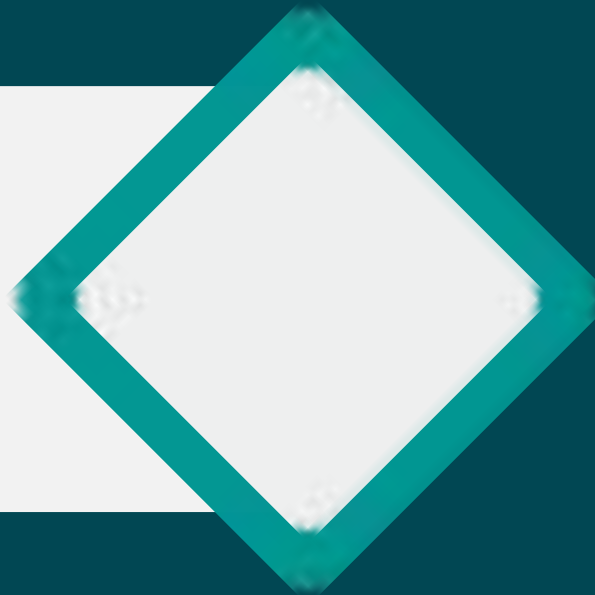
FASE: MANTENIMIENTO

Enfoque tradicional: El mantenimiento comienza después de que el software está en producción.

Enfoque ágil: El mantenimiento es parte del ciclo continuo. Los bugs y las mejoras se gestionan en el backlog y se abordan en iteraciones futuras. Además, el software se actualiza y mejora de manera constante.



INGENIERÍA DE REQUERIMIENTOS



"El arte de entender qué debe hacer el software antes de construirlo."



¿Qué es la Ingeniería de Requerimientos?

Es el proceso de identificar, analizar, documentar y validar las necesidades de los stakeholders para guiar el desarrollo del software.

Si falla aquí → todo lo demás falla.

DATO: El 70% de los proyectos de software fracasan por requerimientos mal definidos. (Standish Group, CHAOS Report)

Características de un buen Requerimiento

- **ESPECÍFICO:** Describe una sola cosa concreta
- **MEDIBLE:** Puede verificarse con una prueba
- **ALCANZABLE:** Técnicamente posible de implementar
- **TRAZABLE:** Vinculado a una necesidad real del negocio
- **SIN AMBIGÜEDAD:** Un solo significado para

EVITAR: "El sistema deberá ser amigable y seguro." (¿Qué significa "amigable"? ¿Cómo medimos "seguro"?)

ETAPAS DE LA INGENIERÍA DE REQUERIMIENTOS



Es un proceso iterativo, no lineal.

CLASIFICACIÓN DE REQUERIMIENTOS



REQUERIMIENTOS FUNCIONALES (RFs)

¿Qué es un Requerimiento Funcional?

Es un requerimiento que define una acción específica que el sistema debe realizar. Responde a: ¿Qué debe hacer el sistema?

EJEMPLOS:

- ✓ El sistema debe permitir al usuario registrarse con correo y contraseña.
- ✓ El sistema debe generar un reporte en PDF de ventas mensuales.
- ✓ El sistema debe enviar un correo de confirmación al completar una compra.

Plantilla formal de Requerimiento Funcional

Referencia: IEEE 830 / ISO 29148

CAMPO	CONTENIDO
ID	RF-001
Función	Nombre corto de la función
Descripción	Qué debe hacer el sistema
Entradas	Datos que recibe el sistema
Fuentes	¿Quién o qué proporciona las entradas?
Salida	Resultado producido
Destino	¿A quién o dónde va la salida?
Acción	Procesamiento interno que ocurre
Precondición	Qué debe ser verdad ANTES de ejecutar
Postcondición	Qué debe ser verdad DESPUÉS
Prioridad	Alta / Media / Baja

Plantilla formal de Requerimiento Funcional

Ejemplo

CAMPO	CONTENIDO
ID	RF-001
Función	Autenticación de usuario
Descripción	El sistema debe permitir al usuario iniciar sesión con correo y contraseña.
Entradas	Correo electrónico, contraseña
Fuentes	Formulario de login (interfaz web)
Salida	Acceso concedido o mensaje de error
Destino	Pantalla principal del sistema
Acción	Validar credenciales contra la base de datos. Si son correctas, crear sesión. Si no, mostrar mensaje de error.
Precondición	El usuario debe estar registrado en el sistema
Postcondición	El usuario tiene sesión activa o se registra el intento fallido
Prioridad	Alta

Plantilla resumida de Requerimiento Funcional

CAMPO	CONTENIDO
ID	RF-001
Descripción	Qué debe hacer el sistema
Actores involucrados	Quiénes interactúan con esta función

CAMPO	CONTENIDO
ID	RF-001
Descripción	El sistema debe permitir al usuario iniciar sesión con correo y contraseña. Si las credenciales son incorrectas, debe mostrar un mensaje de error.
Actores involucrados	Usuario registrado, Sistema

REQUERIMIENTOS NO FUNCIONALES: RESTRICCIÓN (RRs)

Una restricción limita las opciones de diseño o implementación disponibles al desarrollador. Responden a: ¿Qué límites no puede cruzar el sistema?

TIPOS DE RESTRICCIÓN:

- Tecnológica → Debe desarrollarse en Java
- Legal → Debe cumplir con la ley de protección de datos vigente en el país
- Presupuesto → No puede exceder \$10,000 USD
- Interfaz → Debe integrarse con sistema SAP ya existente en la empresa
- Organizacional → Debe seguir los estándares internos del departamento de TI



TECNOLÓGICA

Impone el uso de tecnologías, plataformas o lenguajes específicos.

Ejemplo: el cliente ya tiene un servidor y equipo de soporte en Java.



LEGAL

Obliga al sistema a cumplir con leyes o regulaciones.

Ejemplo: protección de datos personales (LGPD, RGPD).



PRESUPUESTO

Limita los recursos económicos disponibles para desarrollo, infraestructura o licencias.



INTERFAZ

El sistema debe comunicarse o integrarse con otro sistema que ya existe y no puede modificarse.



ORGANIZACIONAL

Políticas internas de la empresa que el sistema debe respetar: estándares de código, flujos de aprobación, herramientas aprobadas.



Plantilla formal de Requerimiento de Restricción

Referencia: IEEE 830

CAMPO	CONTENIDO
ID	RR-001
Nombre	Nombre corto de la restricción
Descripción	Qué limita y por qué
Categoría	Tecnológica / Legal / Presupuesto /Interfaz / Organizacional
Fuente	Quién impone esta restricción
Impacto	Qué áreas del sistema afecta
Verificación	¿Cómo se comprueba que se cumple?
Prioridad	Alta / Media / Baja

Plantilla formal de Requerimiento de Restricción

Ejemplo

CAMPO	CONTENIDO
ID	RR-001
Nombre	Lenguaje de desarrollo backend
Descripción	El sistema debe desarrollarse usando Java 17+ para el backend, ya que el equipo de TI solo da soporte a esta tecnología.
Categoría	Tecnológica / Organizacional
Fuente	Departamento de TI de la institución
Impacto	Arquitectura del servidor, frameworks disponibles, tiempo de desarrollo
Verificación	Revisión del código fuente y del ambiente de despliegue
Prioridad	Alta (no negociable)

Plantilla formal de Requerimiento de Restricción Resumida

CAMPO	CONTENIDO
ID	RF-001
Descripción	Qué limita el sistema
Justificación	Por qué existe esta restricción
Impacto	Qué parte del sistema afecta

CAMPO	CONTENIDO
ID	RF-001
Descripción	El sistema debe desarrollarse en Java 17+ para el módulo backend.
Justificación	El departamento de TI de la institución solo da soporte técnico a Java.
Impacto	Selección de frameworks, herramientas y perfil del equipo de desarrollo

REQUERIMIENTOS NO FUNCIONALES: CALIDAD

Son características que definen QUÉ TAN BIEN el sistema hace lo que tiene que hacer.

También llamados:

- Atributos de calidad
- Requisitos no funcionales de calidad

NO describen una función, describen el nivel de desempeño esperado.

✗ "El sistema debe cargar rápido." → Ambiguo, no medible

✓ "El sistema debe responder en menos de 2 segundos bajo 500 usuarios simultáneos." → Específico, medible

ATRIBUTOS DE CALIDAD DEL SOFTWARE



01. USABILIDAD

Desc: Facilidad de uso e interacción.



02. RENDIMIENTO

Desc: Velocidad de respuesta.



03. DISPONIBILIDAD

Desc: Tiempo activo del sistema.



04. CONFIABILIDAD

Desc: Consistencia y ausencia de fallos.



05. MANTENIBILIDAD

Desc: Facilidad de modificar y reparar.



06. PORTABILIDAD

Desc: Funcionar en distintos entornos.



07. SEGURIDAD

Desc: Protección de datos y accesos.



08. EFICIENCIA

Desc: Uso óptimo de recursos.



09. ESCALABILIDAD

Desc: Soportar crecimiento de carga.



10. INTEROPERABILIDAD

Desc: Comunicación con otros sistemas.



01. USABILIDAD

La facilidad con la que los usuarios pueden interactuar con el sistema.

Aspectos clave:

- Facilidad de aprendizaje
- Eficiencia de uso →
Memorabilidad
- Prevención de errores
- Satisfacción del usuario
- Accesibilidad



02. RENDIMIENTO

La eficiencia del sistema para responder a solicitudes de forma rápida y manejando adecuadamente sus recursos.

Aspectos clave:

- Tiempo de respuesta
- Latencia
- Carga máxima soportada
- Uso de caché y memoria



03. DISPONIBILIDAD

La capacidad del sistema para estar operativo y accesible cuando el usuario lo necesita.

Aspectos clave:

- Tiempo de inactividad (downtime)
- Tolerancia a fallos
- Mantenimientos sin interrupciones

REFERENCIA DE MEDIDA:

99.9% = máx. 8.76 horas/año caído

99.99% = máx. 52 minutos/año caído



04. CONFIABILIDAD

La capacidad del sistema para realizar sus funciones de forma precisa y consistente a lo largo del tiempo.

Aspectos clave:

- Prevención de errores
- Manejo de excepciones
- Recuperación ante fallos

DIFERENCIA IMPORTANTE:

Disponibilidad = ¿El sistema está encendido?

Confiabilidad = ¿El sistema hace bien su trabajo?



05. MANTENIBILIDAD

La facilidad con la que el sistema puede ser modificado, actualizado y reparado.

Aspectos clave:

- Legibilidad del código
- Modularidad y reusabilidad
- Documentación
- Facilidad de pruebas y depuración



06. PORTABILIDAD

La capacidad del sistema para funcionar en diferentes entornos sin modificaciones mayores.

Aspectos clave:

- Compatibilidad entre plataformas
- Independencia tecnológica
- Empaquetado y distribución
- Configurabilidad por entorno

Ejemplo moderno: Docker permite alta portabilidad.



07. SEGURIDAD

La capacidad del sistema para proteger datos, recursos y funcionalidades contra amenazas.

Aspectos clave:

- Confidencialidad (solo quien debe ver, ve)
- Integridad (los datos no son alterados)
- Autenticación (verificar identidad del usuario)
- Auditoría (registrar quién hizo qué y cuándo)



08. EFICIENCIA

La capacidad del sistema para utilizar sus recursos de forma óptima: CPU, memoria, red.

Aspectos clave:

- Optimización de recursos
- Algoritmos eficientes
- Optimización de consultas (queries)
- Tiempo de inicio del sistema

Eficiencia ≠ Rendimiento

Rendimiento = qué tan rápido responde

Eficiencia = qué tan poco consume para lograrlo



09. ESCALABILIDAD

La capacidad del sistema para manejar crecimiento en carga, datos o usuarios sin degradar el rendimiento.

Tipos:

- VERTICAL (más recursos al mismo servidor)
- HORIZONTAL (más servidores en paralelo)

Aspectos clave:

- Desempeño consistente bajo carga alta
- Particionamiento de datos
- Gestión de sesiones distribuidas



10. INTEROPERABILIDAD

La capacidad del sistema para comunicarse e integrarse con otros sistemas externos.

Aspectos clave:

- Compatibilidad de formatos de datos
- APIs abiertas y documentadas
- Estándares de la industria (REST, SOAP, OAuth)
- Integración con sistemas externos
- Compatibilidad de plataformas

Plantilla formal de Requerimiento Plantilla de Atributo de Calidad (Escenario)

Basada en ISO 9126 / ISO 25010

CAMPO	CONTENIDO
ID	EAC-001
Atributo de Calidad	(Cuál de los 10 aplica)
Escenario crudo	Situación real que lo origina
Estímulo	Evento o acción que lo dispara
Fuente del estímulo	Quién o qué genera el evento
Entorno	Condiciones en que ocurre
Artefacto	Componente del sistema afectado
Respuesta esperada	Cómo debe reaccionar el sistema
Medida de la respuesta	SIEMPRE un número medible
Objetivo de negocio	Por qué importa al negocio

Plantilla formal de Requerimiento Plantilla de Atributo de Calidad (Escenario)

Ejemplo

CAMPO	CONTENIDO
ID	EAC-001
Atributo de Calidad	Disponibilidad
Escenario crudo	Estudiante intenta ver sus calificaciones a las 11 PM del cierre de semestre, durante mantenimiento planificado.
Estímulo	Solicitud de acceso al sistema
Fuente del estímulo	Estudiante (usuario final)
Entorno	Operación 24/7 con mantenimientos
Artefacto	Infraestructura (servidores, BD)
Respuesta esperada	Sistema disponible y respondiendo
Medida de la respuesta	99.9% uptime mensual (máx. 43 minutos de downtime/mes)
Objetivo de negocio	Garantizar acceso en períodos críticos del calendario académico

Plantilla formal de Requerimiento Funcional

Ejemplo

CAMPO	CONTENIDO
ID	RF-001
Función	Autenticación de usuario
Descripción	El sistema debe permitir al usuario iniciar sesión con correo y contraseña.
Entradas	Correo electrónico, contraseña
Fuentes	Formulario de login (interfaz web)
Salida	Acceso concedido o mensaje de error
Destino	Pantalla principal del sistema
Acción	Validar credenciales contra la base de datos. Si son correctas, crear sesión. Si no, mostrar mensaje de error.
Precondición	El usuario debe estar registrado en el sistema
Postcondición	El usuario tiene sesión activa o se registra el intento fallido
Prioridad	Alta

CONCEPTOS CLAVE APRENDIDOS

- **Ciclo de Vida del Software**

Comprensión de las fases que recorre un proyecto de software desde su concepción hasta su retiro, incluyendo actividades de aseguramiento de la calidad en cada etapa

.

- **Aseguramiento de la Calidad del Software (SQA)**

Se extiende a lo largo de todo el ciclo de vida del software con el objetivo de prevenir defectos y asegurar el cumplimiento de estándares y requisitos del cliente.

CONCEPTOS CLAVE APRENDIDOS

CHECKLIST — ¿Es un buen requerimiento?

- ¿Tiene un ID único?
- ¿Es específico? (no vago ni general)
- ¿Es medible o verificable con una prueba?
- ¿Es técnicamente alcanzable?
- ¿Está vinculado a una necesidad real del negocio?
- ¿Todos lo interpretan igual?
- ¿Tiene prioridad definida?
- ¿Se puede rastrear hasta el stakeholder que lo pidió?

Si alguna respuesta es NO → debe reescribirse.

REFERENCIAS

<https://www.iso.org/standard/63712.html>

<https://www.iso.org/standard/63711.html>

<https://www.iso.org/standard/72089.html>

<https://ieeexplore.ieee.org/document/8576648>

<https://ieeexplore.ieee.org/document/216199>

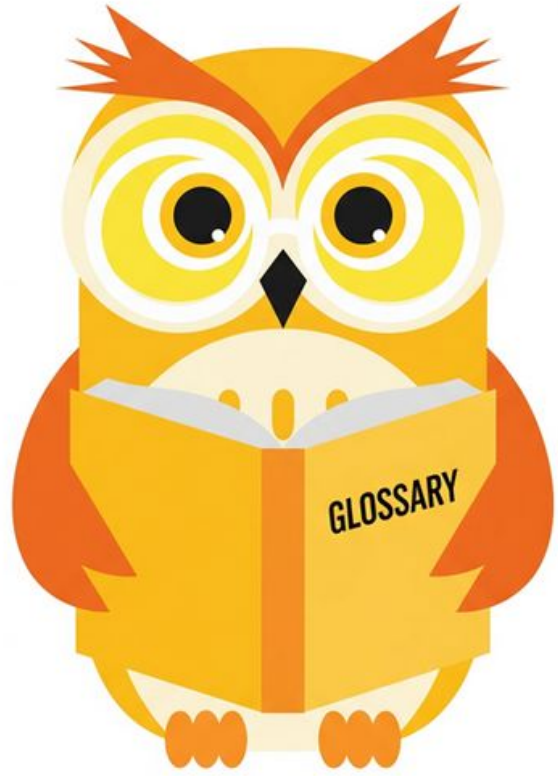
<https://www.iso.org/standard/35733.html>

<https://www.standishgroup.com/>

<https://agilemanifesto.org/n>



Ejemplo práctico



Practiquemos lo aprendido



Nombre de la actividad:

Cantidad de participantes:

Doy fe que esta actividad está
planificada en dtt (Sí/No):

Hora de inicio:

Hora de fin:

Duración (min):

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)

Recursos

- Se colocará el recordatorio de las actividades según corresponda



DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA